

EECS 452 W24 Lab 2

The purposes of this lab are to:

- Work with real-time video
- Perform timing analysis of your code
- Perform color thresholding
- Explore different methods of object tracking

1 Outline

In this lab you will be tracking an orange ball. This involves capturing video, thresholding based on color, and localizing the ball. You will be implementing one method of color thresholding and localization. Additionally this lab will teach you how to time your code, which is important for systems that must operate in real-time scenarios. You have been given a skeleton file with the general structure of the lab. The parts that you will fill in have been identified with `TODO` comments.

Pre-Lab

Submit the answers to the Pre-Lab questions to the Pre Lab 2 assignment in Canvas.

In the Pre-Lab, you will be using Matlab to learn about file I/O and two standard representations of colored images. There are many parallels between Matlab's Image Processing Toolbox and OpenCV, so as you learn within a Matlab/PC environment, you will be ahead of the game when transferring your problem to the OpenCV/PI environment. In fact, during and following Lab 2, you will be encouraged to go between each programming environment as you solve a problem in image representation. As you will see, Matlab is often a better choice for prototyping a solution before coding the solution on the R-Pi.

Background Reading

To get started, launch Matlab. You can open the documentation using the `doc` command. In the documentation, open up the Image Processing Toolbox and work through the following Document pages:

- Basic Image Import, Processing, and Export. (Use the search bar if you are having difficulty finding the specific document page.)
- Read the documentation on `imread`. Keep track of the data formats that Matlab supports. Think about: What is the internal representation of images in Matlab? Does this representation depend on whether the external image is gray-scale, bw, or color?
- For more information on reading images, read the documentation on Read Image Data into the Workspace
- Check out the variety of I/O functions for images. See Read and Write Image Data from Files in the documentation.
- Matlab supports some useful interactive tools for exploring an image. Read Display and Exploration to make Pre-Lab 2 a bit easier.
- Finally, a variety of standards exist for representing color. In the Prelab, you will be using Matlab's `rgb2hsv` function to convert one representation (Red, Green, Blue) of a color image to another (Hue, Saturation, Value). Read Color in the Documentation. Although very brief, this page contains links

for further reading in Matlab's documentation. Wikipedia entries for HSV, Color Space, and RGB contain much more information that you will find useful.

Pre-Lab Assignment

Using Matlab, obtain RGB and HSV values for the images found in the Lab 2 directory on Canvas and answer the questions below. (Note: You do not need to submit the specific RGB and HSV values. Matlab will output very large arrays. Only look for trends in these arrays.)

- Q1.** Based on your reading above, what are RGB and HSV values and what are they used for?
- Q2.** Suppose two colors differ in HSV space in the V dimension. How do these two colors differ visually?
- Q3.** Suppose two colors differ in RGB space in the R dimension. How do these two colors differ visually?
- Q4.** Obtain histograms of RGB and HSV values of the mid light image and qualitatively describe how the histogram of each channel (R,G,B,H,S,V) is different.

With respect to the three different lighting conditions:

- Q5.** How do the RGB values of the scene change as light conditions change from min, mid to max?
- Q6.** How do the HSV values of the scene change as light conditions change from min, mid to max?
- Q7.** Do these changes you observe agree with what you've read about the two color spaces? Explain your answer.

In-Lab

2 Video capture

Download the starter files `lab2.picamera2.py` and `timer.py`, ensuring that the files are located in the same directory. The skeleton code has basic video capture already implemented using a circular buffer, loosely based on [this picamera2 example](#). Setup the camera and run the code in the skeleton file. It will display a video feed from the camera. You can quit by pressing 'q'. Refer to the skeleton code or the tutorial to answer the following questions:

- Q8.** What are some possible benefits of capturing images to a circular buffer, versus capturing an image using `(picam2.capture_image())` every iteration of the loop?
- Q9.** What does the 1 signify in `cv2.waitKey(1)`? Can you tell OpenCV to wait for less time?

Edit the code so that when you press the 'c' key it plots the current frame using pyplot's `imshow` function. Pyplot's `imshow` will pause the video feed, so if you would like to later plot real-time images in addition to the Raw Image, create `cv2` plots.

- Q10:** Why are the colors in the displayed image wrong? Hint: What is OpenCV's ordering for the color channels?

Fix the coloring issue in your image capture function (Hint: Look up `cvtColor`). Capture an image of your orange ball. Note that as you mouse over the image, pyplot will show the color values for the pixel that your mouse is over.

- Q11.** What is a typical RGB pixel value for the orange ball?
- Q12.** In the report, show a screenshot of your working video capture and image capture code, with both the OpenCV and pyplot images displayed.

3 Color Thresholding

For the color-thresholding method implemented in this lab, you will be generating a difference image based on how different each pixel is from a specified color. The choice of color space and distance metric can have a large impact on the resulting difference image. You will base your difference image on the H-channel of the HSV colorspace since it is designed to indicate the hue of a pixel.

3.1 H-value

Find a typical H-value for your orange ball as a starting point for your difference image calculation. Assign it to the variable `H_val` in the **CONSTANTS AND PARAMETERS** section of the code. This value may be adjusted later. You may base it on your typical RGB value from the previous section, or you may modify your image capture code to display HSV pixel values and recapture the data.

Q13. What is your chosen H-value?

Fill in the function `color_subtract` which takes as input an image and your chosen H-value and returns the difference image where each pixel is the absolute value of the difference between the chosen H-value and that pixel of the input image. Use this to calculate the difference image of every frame and display it in a separate window. There is a **TODO** marker in the code where you should be plotting the image. **Note:** The H-channel pixel values are unsigned 8-bit integers, so you must cast them to a sufficiently large signed type to avoid overflow during your calculations. Numpy's `astype` member function may be useful here.

3.2 Threshold

You need to convert the difference image to a binary image in order to localize the ball. In the binary image, the ball should be white while everything else is black. Use OpenCV's `threshold` function to convert the difference image into a binary image. Use the `thold_val` variable to set the `thresh` parameter to something small, such as 0. You will tune it and the H-value parameter later. Answer the questions below to determine the `type` parameter.

Q14. What color is the ball in your difference image?

Q15. What is the appropriate `type` parameter to pass to OpenCV's `threshold` function to make the ball white while keeping other pixels black in the binary image?

3.3 Box Filter

You may notice a lot of background noise in your binary image. This can be reduced with a box filter. A box filter averages the pixel values over an area, thus functioning as a low-pass filter. Use OpenCV's `boxFilter` function to apply a box filter to your difference image before thresholding. You only need to be concerned about the `src`, `ddepth`, and `ksize` parameters. Set `ddepth` to -1. A `ksize` of (5,5) should be sufficient, though you may experiment with other values.

Q16. What is the shape of the difference image after calling the `boxFilter` function? Use Numpy's `shape` method.

3.4 Trackbars

It is often necessary to tune algorithm parameters when designing a computer vision application. OpenCV allows for the creation of trackbars that can be used to control algorithm parameters during execution thus removing the need to iteratively edit parameters and re-run the application. Follow the example at:

https://docs.opencv.org/4.5.2/da/d6a/tutorial_trackbar.html

to create two trackbars, one for H-value and one for the threshold parameter. You will need to add code in the **TRACKBARS** section to create the `trackbar` window and the individual trackbars. You will also need

to edit code in the main loop to read the values from the trackbars during program execution and pass the values to your color thresholding algorithm.

Use these to tune your algorithm so that as much as possible the binary image only shows where the ball is. You may make another trackbar for adjusting the kernel size of the box filter function, though this is not required.

Q17. Briefly explain each of the five arguments passed to `createTrackbar`.

Q18. What values do you get for the H-value and the threshold after tuning them?

Q19. Show in the report an example binary image you obtained.

4 Timing

Reducing latency is an important aspect of real-time computer vision and has been identified by past EECS 452 students as one of the more difficult parts of their team projects. To reduce latency, it is key to understand the timing of different parts of your code. Often there is a tradeoff between more accurate, slower algorithms and less accurate, faster algorithms.

The file `timer.py` contains skeleton code for a class named `Timer`. Each instance of the class has the ability to reset a timer and to print the time elapsed since the last reset. You will see three different usages of the `Timer` class already in the code provided to you. These instances measure the times of color thresholding, ball localization, and image display.

Fill in the missing code in `timer.py` to make the timers functional. You will need to use Python's `time.time()` function. You may refer to `frame_start` and `frame_end` in the code given to you which keep track of the total processing time of each frame without using the `Timer` class. Answer the questions below.

Q20. List the execution times for the following parts of your algorithm. The timing will vary slightly between frames, so you may put typical times, a calculated average, or a range.

1. Total time to perform color thresholding
2. Time to display images
3. Time to process one frame

Q21. How many frames per second do you currently process?

Add instances of the `Timer` class to time each of the following three parts of the color thresholding: difference image generation, box filtering, thresholding.

Q22. List the execution times for the following parts of your algorithm:

1. Time to calculate the difference image
2. Time to run box filter
3. Time to threshold image

5 Ball Localization

The binary image is used to determine the size and location of the ball. The first method you implement will be based on a heuristic. Under the assumption that the ball is the only object remaining in the threshold image, the centroid of all white pixels in the binary image should be the center of the ball. Likewise, the total number of white pixels in the input image should correspond to the ball's area, thus allowing the radius to be calculated.

Recall the centroid is the arithmetic mean of the points of the object. It can be calculated as

$$C_x = \sum_i^k \frac{x_i}{k}, \quad C_y = \sum_i^k \frac{y_i}{k}$$

where C_x and C_y are the x- and y-coordinates of the centroid; x_i and y_i are the x- and y-coordinates of the i th white point; and k is the number of white points in the image. k also represents the area of the region and is used to approximate the ball's radius. (Recall that the area of a circle is $A = \pi r^2$.)

The `identify_ball` function is a function called in the main loop that you will program to take the binary image from color thresholding and return the center and radius of the ball using the heuristic described above. Fill in the section of the code that uses a double nested for loop to calculate C_x , C_y , and k . Use those to calculate and return the center and radius of the ball.

The main loop is set up to call the `identify_ball` function and to draw the resulting detected ball on the original image. Be sure that the `USE_LOCALIZATION_HEURISTIC` flag is set to `True` and that the `USE_IDX_IMG` flag is set to `False` before running your code.

Q23. How long does the ball localization process take? How many frames per second can you process?

The above implementation is clearly too slow to be useful. As an interpreted language, Python is relatively slow. To improve speed, it is necessary to take advantage of precompiled libraries. Numpy offers optimized linear algebra functions, so converting centroid calculation to a linear algebra problem would allow you to use it to speed up the computation.

One method to do this is to use the index image. An index image has the same dimensions as the binary image, but the value at each pixel is either the x- or y-index of that pixel. By masking the index image with the binary image, the value of each white pixel will be its x- or y-coordinate while the value of each black pixel will be zero. All pixels can then be summed and divided by k to yield C_x and C_y . Since these coordinates each only change in one dimension, each row (in the x-index image) or column (in the y-index image) is the same. Thus instead of doing an element-wise multiply between the binary image matrix and the full index image matrix and then summing the values, you can save memory by performing a matrix-vector multiplication between the binary image matrix and the single repeated row (for x) or column (for y) of the index image and then summing the values.

Fill in the section of code in `identify_ball` that uses the index image for calculation. Compute the center and radius as described above. The variables `x_idx` and `y_idx` are given to you which represent the repeated row of the x-index image and the repeated column of the y-index image respectively. Set the `USE_IDX_IMG` flag to `True` before running your code. **Note:** The white pixels have in the binary image have a value of 255, not 1, so you may have to account for that in your calculations.

Q24. How long does the ball localization process take using the index image method? How many frames per second can you process?

Q25. Show in the report (using a supplementary file such as video capture if needed) that your implementation of ball localization using the index image method actually works.

6 Post-Lab

Q26. Write a few sentences or a short paragraph comparing and contrasting the two ball localization methods. Take into account both the quantitative timing results and qualitative accuracy observations.

Q27. Python and C++ are programming languages that have significant differences. We have listed three major differences between these languages below. Please answer the questions corresponding to each. The following reference may be helpful: <https://www.python.org/doc/essays/comparisons/>

- A) What are some differences between interpreted and compiled programming languages? What type of language is Python? What type of language is C++?

- B) What does it mean for a programming language to be dynamically typed versus statically typed? How is Python typed? How is C++ typed?
- C) What are some pros and cons of garbage collection? Does Python implement garbage collection? What about C++?

Q28. Answer the following questions about using C++ OpenCV on Raspberry Pi. This tutorial is a useful reference: https://docs.opencv.org/4.6.0/db/df5/tutorial_linux_gcc_cmake.html

- A) How do you import openCV in C++?
- B) How do you compile C++ code in a Raspbian environment?

Q29. Read the following comparisons about C++ vs Python performance for OpenCV and answer the questions that follow.

- Top answer of this StackOverflow question: <https://stackoverflow.com/questions/13432800/does-performance-differs-between-python-or-c-coding-of-opencv>.
 - LearnOpenCV post comparing MATLAB to OpenCV for C++/Python (the C++ vs. Python comparison is towards the end of the article): <https://www.learnopencv.com/opencv-c-vs-python-vs-matlab-for-computer-vision/>
- A) What causes OpenCV Python implementations to be slower than C++ implementations of the same code?
- B) Why might you choose to code in Python anyway?

7 Deliverables

Post your report with a typed set of answers to the questions. Post supplementary files (captured video, pictures, code, etc.) as necessary. If all the deliverables (screenshots, etc.) can be put into a single PDF file, then a single PDF file is an acceptable submission.